

# GOLD-001 — The Complete Record

Production RAG v1 · six closed batches · 90 holdout-eligible cases · corpus snapshot  
snap\_689e336380a054d8039dc35b2c09cd0a · status generated 2026-08-24T17:47:23Z

WHERE THE PROJECT STANDS

90 human-verified cases, all 90 of them holdout-eligible, drawn from 99 candidates across six closed batches. 9 were rejected and every one is kept as a negative audit example. 1 case is genuine multi-hop.

No retrieval system has ever been run against a GOLD candidate. No holdout and no validation split is frozen. SYSTEM-A and SYSTEM-B remain frozen and unexecuted.

90  
human-verified

90  
holdout-eligible

9  
rejected, all preserved

60  
short of the 150 target

1. The six closed batches

2. What is in the benchmark

3. Every approved case

4. Every rejection

5. Batch 004 — multi-hop, measured

6. Batch 005 — two lanes

7. Batch 006 — the four fixes, and the shortfall

8. The generator defect ledger

9. Modules built

10. Heading parser audit

11. Batch 007 — preregistered

12. Tests and invariants

13. Commits and artifacts

## 1. The six closed batches

A batch closes only when every candidate has a final human decision. The closure records a hash over its candidate records, and the test suite re-checks that hash, so an edit after closure fails the tests rather than passing unnoticed.

| batch | closed     | candidates | verified | rejected | acceptance | eligible | closure hash  |
|-------|------------|------------|----------|----------|------------|----------|---------------|
| 001   | 2026-08-20 | 18         | 16       | 2        | 89%        | 16       | d6f92e8d1a... |
| 002   | 2026-08-20 | 18         | 17       | 1        | 94%        | 17       | 69364f672e... |
| 003   | 2026-08-21 | 20         | 20       | 0        | 100%       | 20       | e186a2f30e... |
| 004   | 2026-08-23 | 15         | 14       | 1        | 93%        | 14       | 57269018d2... |
| 005   | 2026-08-24 | 19         | 15       | 4        | 79%        | 15       | ffbf9dda40... |
| 006   | 2026-08-25 | 9          | 8        | 1        | 89%        | 8        | 7ff5596c75... |
| all   |            | 99         | 90       | 9        | 91%        | 90       |               |

THREE STATES, DELIBERATELY KEPT APART

precheck\_holdout\_ready is **structural only** — a script's verdict that a record is checkable. human\_verified is an **owner decision** and nothing else can set it. holdout\_eligible is **derived**: a deterministic gate re-run against the current records at every closure.

The distinction is load-bearing. Batch 005 shipped 19 of 19 precheck-ready and its review still repaired seven and rejected four. Batch 006 shipped 9 of 9 precheck-ready and the owner still rejected one and revised five.

## 2. What is in the benchmark

53

distinct source documents

54 / 36

anthropic / openai

82 / 8

single-span / multi-span

| reasoning type                    | cases | share |
|-----------------------------------|-------|-------|
| exact_lookup                      | 19    | 21%   |
| configuration_interaction         | 14    | 16%   |
| error_behavior                    | 14    | 16%   |
| lifecycle_compatibility_migration | 6     | 7%    |
| lifecycle                         | 2     | 2%    |
| ambiguity_disambiguation          | 1     | 1%    |
| genuine_multi_hop                 | 1     | 1%    |

Every case is anchored to exact character offsets in the frozen corpus, carries an evidence hash, and lists the literal critical strings its claims depend on. Ground truth never depends on chunk IDs, so a chunking change cannot silently invalidate the benchmark.

### 3. Every approved case

All 90 human-verified cases, in id order. Batch 001's rows come from its v2 overlay, which supersedes the closed v1 record for eligibility.

| id           | prov | reasoning type           | question |
|--------------|------|--------------------------|----------|
| GOLD-B001-01 | anth | <input type="checkbox"/> |          |
| GOLD-B001-02 | open | <input type="checkbox"/> |          |
| GOLD-B001-03 | anth | <input type="checkbox"/> |          |
| GOLD-B001-04 | open | <input type="checkbox"/> |          |
| GOLD-B001-05 | anth | <input type="checkbox"/> |          |
| GOLD-B001-06 | open | <input type="checkbox"/> |          |
| GOLD-B001-07 | anth | <input type="checkbox"/> |          |
| GOLD-B001-08 | open | <input type="checkbox"/> |          |
| GOLD-B001-09 | anth | <input type="checkbox"/> |          |
| GOLD-B001-10 | open | <input type="checkbox"/> |          |
| GOLD-B001-11 | anth | <input type="checkbox"/> |          |
| GOLD-B001-12 | open | <input type="checkbox"/> |          |
| GOLD-B001-13 | anth | <input type="checkbox"/> |          |
| GOLD-B001-14 | anth | <input type="checkbox"/> |          |
| GOLD-B001-17 | anth | <input type="checkbox"/> |          |
| GOLD-B001-18 | anth | <input type="checkbox"/> |          |
| GOLD-B002-01 | anth | <input type="checkbox"/> |          |
| GOLD-B002-02 | anth | <input type="checkbox"/> |          |
| GOLD-B002-03 | open | <input type="checkbox"/> |          |
| GOLD-B002-04 | anth | <input type="checkbox"/> |          |
| GOLD-B002-05 | anth | <input type="checkbox"/> |          |
| GOLD-B002-06 | anth | <input type="checkbox"/> |          |
| GOLD-B002-07 | anth | <input type="checkbox"/> |          |
| GOLD-B002-08 | anth | <input type="checkbox"/> |          |
| GOLD-B002-09 | anth | <input type="checkbox"/> |          |
| GOLD-B002-10 | anth | <input type="checkbox"/> |          |
| GOLD-B002-11 | anth | <input type="checkbox"/> |          |
| GOLD-B002-13 | anth | <input type="checkbox"/> |          |
| GOLD-B002-14 | anth | <input type="checkbox"/> |          |
| GOLD-B002-15 | open | <input type="checkbox"/> |          |

| id           | prov | reasoning type             | question                                                                                                               |
|--------------|------|----------------------------|------------------------------------------------------------------------------------------------------------------------|
| GOLD-B002-16 | anth |                            |                                                                                                                        |
| GOLD-B002-17 | open |                            |                                                                                                                        |
| GOLD-B002-18 | anth |                            |                                                                                                                        |
| GOLD-B003-01 | anth | configuration_interaction  |                                                                                                                        |
| GOLD-B003-02 | anth | configuration_interaction  |                                                                                                                        |
| GOLD-B003-03 | anth | error_behavior             |                                                                                                                        |
| GOLD-B003-04 | anth | error_behavior             |                                                                                                                        |
| GOLD-B003-05 | anth | error_behavior             |                                                                                                                        |
| GOLD-B003-06 | anth | error_behavior             |                                                                                                                        |
| GOLD-B003-07 | anth | exact_lookup               |                                                                                                                        |
| GOLD-B003-08 | anth | exact_lookup               |                                                                                                                        |
| GOLD-B003-09 | anth | exact_lookup               |                                                                                                                        |
| GOLD-B003-10 | anth | lifecycle                  |                                                                                                                        |
| GOLD-B003-11 | anth | lifecycle                  |                                                                                                                        |
| GOLD-B003-12 | anth | exact_lookup               |                                                                                                                        |
| GOLD-B003-13 | open | configuration_interaction  |                                                                                                                        |
| GOLD-B003-14 | open | configuration_interaction  |                                                                                                                        |
| GOLD-B003-15 | open | exact_lookup               |                                                                                                                        |
| GOLD-B003-16 | open | exact_lookup               |                                                                                                                        |
| GOLD-B003-17 | open | exact_lookup               |                                                                                                                        |
| GOLD-B003-18 | open | exact_lookup               |                                                                                                                        |
| GOLD-B003-19 | open | exact_lookup               |                                                                                                                        |
| GOLD-B003-20 | open | exact_lookup               |                                                                                                                        |
| GOLD-B004-01 | anth | error_behavior             | When using server tools, what does the API return if the server-side sampling loop reaches its i                       |
| GOLD-B004-02 | anth | configuration_interaction  | What happens if Claude calls web search and one of your client tools in the same group of parall                       |
| GOLD-B004-03 | anth | error_behavior             | What happens if the most recent assistant message contains <code>thinking</code> or <code>redacted_thinking</code> blo |
| GOLD-B004-04 | anth | error_behavior             | On Claude 4.5 models and newer, if input tokens plus <code>max_tokens</code> exceed the context window size            |
| GOLD-B004-05 | anth | error_behavior             | What happens if Claude attempts more searches than allowed?                                                            |
| GOLD-B004-06 | anth | exact_lookup               | Within the <code>user_location</code> parameter, what value does the <code>timezone</code> field take?                 |
| GOLD-B004-07 | anth | lifecycle_compatibility_mi | If I need a hard ceiling on thinking costs, what is the support status of extended thinking with                       |

| id           | prov | reasoning type             | question                                                                                                                                       |
|--------------|------|----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| GOLD-B004-09 | open | ambiguity_disambiguation   | In a <code>FunctionToolCallArgumentsDeltaEvent</code> , what does the <code>parsed_arguments</code> field contain, and                         |
| GOLD-B004-10 | open | configuration_interaction  | <code>RealtimeRunner</code> uses <code>OpenAIRealtimeWebSocketModel</code> by default. What happens if I pass a differ                         |
| GOLD-B004-11 | open | error_behavior             | In a streamed run, what happens if a tool requires approval?                                                                                   |
| GOLD-B004-12 | open | configuration_interaction  | What happens if you are manually continuing from <code>result.to_input_list(mode="normalized")</code> , and                                    |
| GOLD-B004-13 | open | exact_lookup               | For a handoff, what does the <code>input_type</code> option specify?                                                                           |
| GOLD-B004-14 | open | exact_lookup               | In the <code>handoff()</code> function, what does the <code>input_filter</code> option do?                                                     |
| GOLD-B004-15 | open | genuine_multi_hop          | If I set <code>needs_approval</code> to <code>True</code> on a function tool used with hosted agents, what happens?                            |
| GOLD-B005-02 | anth | configuration_interaction  | What happens when <code>jwt.type</code> is <code>discovery</code> and no <code>discovery_base</code> is set?                                   |
| GOLD-B005-03 | anth | error_behavior             | What happens when filtering by a non-existent <code>deployment_id</code> ?                                                                     |
| GOLD-B005-04 | anth | error_behavior             | What happens if <code>encrypted_content</code> is missing or modified?                                                                         |
| GOLD-B005-05 | anth | exact_lookup               | What constraint does a request-level <code>allowed_domains</code> list have to satisfy?                                                        |
| GOLD-B005-07 | anth | lifecycle_compatibility_mi | Is <code>compaction_control</code> still supported?                                                                                            |
| GOLD-B005-08 | anth | lifecycle_compatibility_mi | Is manual extended thinking with <code>budget_tokens</code> supported on Claude Sonnet 5?                                                      |
| GOLD-B005-09 | open | error_behavior             | What happens if the arguments are malformed JSON, are valid JSON but not an object (for example,                                               |
| GOLD-B005-11 | open | configuration_interaction  | When using the <code>bedrock(...)</code> provider, what does setting <code>AWS_BEDROCK_BASE_URL</code> change?                                 |
| GOLD-B005-12 | open | configuration_interaction  | What does <code>FuseMountPattern</code> require?                                                                                               |
| GOLD-B005-14 | open | configuration_interaction  | What does <code>ApplyPatchTool</code> require?                                                                                                 |
| GOLD-B005-15 | open | configuration_interaction  | When a <code>ComputerTool</code> is present, how are <code>tool_choice="computer"</code> , <code>"computer_use"</code> and <code>"compu</code> |
| GOLD-B005-16 | open | configuration_interaction  | When you use a <code>Session</code> such as <code>SQLiteSession</code> , what usage does each call to <code>Runner.run(...)</code>             |
| GOLD-B005-17 | open | error_behavior             | What happens if we exceed the <code>max_turns</code> passed?                                                                                   |
| GOLD-B005-18 | open | exact_lookup               | What must an Undici-specific option like <code>dispatcher</code> be paired with?                                                               |
| GOLD-B005-19 | open | lifecycle_compatibility_mi | What happened to <code>httpAgent</code> ?                                                                                                      |
| GOLD-B006-01 | anth | exact_lookup               | What credential is required to create an <code>admin</code> -role service account, and what role may a workl                                   |
| GOLD-B006-02 | anth | error_behavior             | What happens if <code>role: "system"</code> is used in <code>messages</code> with Claude Opus 4.7?                                             |
| GOLD-B006-03 | anth | lifecycle_compatibility_mi | How do the newer code-execution tool versions behave on Claude Haiku 4.5?                                                                      |
| GOLD-B006-04 | anth | exact_lookup               | What effort level does Claude Sonnet 5 default to on the Claude API and Claude Code?                                                           |

| id           | prov | reasoning type             | question                                                                                                             |
|--------------|------|----------------------------|----------------------------------------------------------------------------------------------------------------------|
| GOLD-B006-05 | anth | exact_lookup               | What does <code>thinking.display</code> default to on <code>claude-mythos-5</code> and <code>claude-fable-5</code> ? |
| GOLD-B006-07 | open | configuration_interaction  | What does Setting <code>audio.input.turn_detection</code> to <code>None</code> disable?                              |
| GOLD-B006-08 | open | lifecycle_compatibility_mi | What does the OpenAI Python SDK's temporary legacy-client compatibility path require, and how sh                     |
| GOLD-B006-09 | open | exact_lookup               | What does <code>RunErrorHandlerResult.include_in_history</code> default to?                                          |

## 4. Every rejection

All 9 rejected candidates are kept in their batch records as negative audit examples. None was deleted, and none was replaced with a substitute to keep a count up.

| id           | batch | why                                                                                                                                                                                                                  |
|--------------|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| GOLD-B001-15 | 001   | REJECT — code/example false binding. <code>required: ["location"]</code> belongs to the tool input schema in the example request and does not establish a general requirement on <code>tool_choice</code> .          |
| GOLD-B001-16 | 001   | REJECT — concerns the implementation behaviour of a local example helper function rather than an externally meaningful API or documentation fact. Stronger benchmark material is preferred.                          |
| GOLD-B002-12 | 002   | REJECT — OVERSIZED_EVIDENCE_ANCHOR / EVIDENCE_PRECISION_COST. The documentation is correct and the fact is real. Making the anchor self-contained required expanding it from ~70 to ~1,430 cha                       |
| GOLD-B004-08 | 004   | Rejected: BENCHMARK_QUALITY / TAXONOMY_MISCLASSIFICATION. The facts are not disputed — ContentDeltaEvent carries type "content.delta" and ContentDoneEvent carries type "content.done" — but t                       |
| GOLD-B005-01 | 005   | Rejected: BENCHMARK_QUALITY / TAXONOMY_MISCLASSIFICATION / SCOPE_DEFECT. <code>invalid_tool_input</code> is the same semantic concept in both sources — the tool input is invalid — and the examples diff            |
| GOLD-B005-06 | 005   | Rejected: UNANSWERABLE_AS_ASKED / CATEGORY_MISCLASSIFICATION. The evidence is a schema description of what model IDs may appear in <code>fallbacks[i].model</code> and what an empty list means. It does             |
| GOLD-B005-10 | 005   | Rejected: RELATION_DIRECTION / UNRECOVERABLE_SCOPE. The evidence says the experimental model rejects caller-supplied <code>betas</code> overrides; it does not say <code>betas</code> overrides anything, so the gen |
| GOLD-B005-13 | 005   | Rejected: DUPLICATE_RELATION / BENCHMARK_REDUNDANCY. The fact is supported — <code>S3FilesMountPattern</code> requires broad acknowledgement because <code>mount.s3files</code> uses ambient IAM authority — but it  |
| GOLD-B006-06 | 006   | REJECT — DUPLICATE_FACT / BENCHMARK_REDUNDANCY. The fact is supported, but GOLD-B005-11 already carries substantially the same operational relation from the OpenAI Python library; this obtai                       |

## 5. Batch 004 — multi-hop, measured rather than assumed

Batch 004 set out to find genuinely composable multi-hop questions: cases where the answer requires combining two spans and neither span alone suffices. It tested **559** identifier-sharing pairs and found **1**.

| why a pair was rejected                              | pairs |
|------------------------------------------------------|-------|
| span 1 alone answered the full question              | 16    |
| span 2 alone answered the full question              | 16    |
| the spans were two unrelated lookups                 | 379   |
| no bridge relationship existed                       | 147   |
| the composed answer introduced unsupported inference | 0     |
| rejected                                             | 558   |
| passed                                               | 1     |

A near-miss diagnostic re-examined the 5 pairs that failed on a single gate. All were **correct rejections** — the rule under test was *states\_dependency* — *span 2's conditional must test the bridge entity's own state*. The best-known of them, `max_tokens`, is a request parameter in one span and a `stop_reason` value in the other: the same token, two different things. That near-miss is why `rag_v1.gold.bridge_equivalence` exists, and it is now a regression test.

Batch 004 closed at 14 of 15 with 6 anchor repairs and 3 owner overrides. An override never deletes a finding: the finding stays on the record next to the owner's decision to accept it.

## 6. Batch 005 — two lanes, and a second multi-hop measurement

Batch 005 split its effort. Lane A spent it where the corpus pays — interactions, constraints, lifecycle statements, cross-component ambiguity. Lane B searched for chains **dependency-first**, opening only on sentences that state a dependency, rather than batch 004's all-pairs sweep.

| dependency-first search     | count |
|-----------------------------|-------|
| dependency pairs considered | 3     |
| failed span independence    | 1     |
| failed semantic equivalence | 1     |
| passed                      | 1     |
| budget                      | 1000  |
| new unique chains exported  | 0     |

### TWO SEARCHES, TWO METHODS, ONE CHAIN

The single valid chain batch 005 found was the chain batch 004 had already closed, so batch 005 exported none. **That is a result about the corpus, not a failed search:** this frozen snapshot contains very little naturally composable multi-hop structure. It is why batch 006 was instructed not to search again, and why the project's multi-hop count is 1 and is not presented as coverage.

Batch 005 targeted 30 and exported 19; of the 46 candidates that reached the semantic self-review, 27 were dropped. It closed at 15 of 19 — the lowest acceptance rate of the six (79%), and the batch that produced four of the seven generator defects in the ledger below.

## 7. Batch 006 — the four fixes, and a shortfall worth having

Batch 006 implemented the four fixes batch 005's closure preregistered, **before** authoring anything, then targeted 28 candidates and exported 9.

| the census behind the shortfall          | count |
|------------------------------------------|-------|
| facts mined                              | 1361  |
| distinct evidence spans the miners reach | 773   |
| of those, unspent by any closed batch    | 699   |
| mined facts that reached no builder      | 2482  |

THE FINDING THAT CHANGED THE PLAN

**The corpus is not exhausted. The authoring is.** 699 distinct evidence spans in the frozen snapshot have never been used by any closed batch, and no deterministic template could turn them into a question without paraphrasing — what remains is long, multi-clause prose, and a template that fits it is a template that invents wording.

Refusing all paraphrase is precisely what keeps those facts out of the benchmark. That is the problem batch 007 is preregistered to solve.

The owner approved 8 of 9: five after revisions they specified, three as generated, one rejected. Three taxonomy corrections were applied — a requirement mislabelled as a configuration interaction, a compatibility statement as an exact lookup, a migration note as an interaction — and **no anchor moved**, which a test now enforces. Every revision is attributed to the owner rather than to Claude's review, because a change someone asked for and a change the author proposed are different acts.

### The one rejection, and the gate that missed it

`GOLD-B006-06` was rejected as **DUPLICATE\_FACT / BENCHMARK\_REDUNDANCY**. The fact is supported, but `GOLD-B005-11` already carries the same operational relation from the OpenAI Python library while this obtained it from the TypeScript library. Duplicate control compares question text, span offsets and span text — and two libraries documenting the same behaviour share none of those. Recorded as defect E.

## 8. The generator defect ledger

Every defect a review found in the generator, what case exposed it, and where it was or will be fixed. A defect is **recorded, never patched into a closed batch**: the generation artifact stays as generated, and the fix belongs in the next batch's preregistration where it can be declared before it sees a candidate.

| defect                                                                                    | seen in                   | found            | fixed in                  |
|-------------------------------------------------------------------------------------------|---------------------------|------------------|---------------------------|
| <b>A</b> the bare-definition-bullet rule only inspects single-span records                | <code>GOLD-B005-01</code> | batch 005 review | batch 006                 |
| <b>B</b> markdown reference links survive into questions built from conditional sentences | <code>GOLD-B005-15</code> | batch 005 review | batch 006                 |
| <b>C</b> prose mistaken for a section heading by the parser                               | <code>GOLD-B005-11</code> | batch 005 review | batch 006                 |
| <b>D</b> subject–relation direction must remain explicitly checked                        | <code>GOLD-B005-10</code> | batch 005 review | batch 006                 |
| <b>E</b> cross-library duplicate facts are invisible to duplicate control                 | <code>GOLD-B006-06</code> | batch 006 review | batch 007 (preregistered) |

|   | defect                                                              | seen in                                  | found            | fixed in                  |
|---|---------------------------------------------------------------------|------------------------------------------|------------------|---------------------------|
| F | compound single-span facts are labelled by their first verb         | GOLD-B006-01, GOLD-B006-03, GOLD-B006-08 | batch 006 review | batch 007 (preregistered) |
| G | questions inherit the breadth of their frame, not of their evidence | GOLD-B006-02, GOLD-B006-04, GOLD-B006-05 | batch 006 review | batch 007 (preregistered) |

#### A. the bare-definition-bullet rule only inspects single-span records

The generation self-review drops a candidate whose single span is a bare '`field`: description' bullet, because its scope lives in the heading. B005-01 is two such bullets in a multi-span record, and the rule's `len(spans) == 1` guard let it through. Recorded rather than patched: batch 005's generation artifact is not being regenerated, and a fix belongs in batch 006's preregistration.

**Fix:** Extend the bare-definition-bullet scope rule to every span of a multi-span record, not only records with exactly one span.

#### B. markdown reference links survive into questions built from conditional sentences

The link stripper runs on the composed question, but a reference-style link whose label is itself backticked was not matched. Repaired here by hand; the pattern should be fixed before batch 006.

**Fix:** Strip markdown reference links whose label is itself backticked, before a question is composed.

#### C. prose mistaken for a section heading by the parser

`section_path` reads 'configured through AWS\_REGION, AWS\_DEFAULT\_REGION, or your AWS profile.' No claim depends on it, and closed batches must not be touched, so this is recorded as a parser observation for a future audit.

**Fix:** Audit the heading parser against the corpus snapshot and record how often ordinary prose is captured as a `section_path`. Do NOT fix historical parser output in place; closed batches keep what they have.

#### D. subject-relation direction must remain explicitly checked

Rejected: `RELATION_DIRECTION / UNRECOVERABLE_SCOPE`. The evidence says the experimental model rejects caller-supplied `betas` overrides; it does not say `betas` overrides anything, so the generated question reverses the documented relation. Rewriting around the true subject would require identifying the experimental model, whose identity is outside the exact evidence and cannot be recovered by a minimal valid expansion. Not salvaged.

**Fix:** Every generated question must be re-read against its span for relation direction: the question's subject must be the source's subject, not its object.

#### E. cross-library duplicate facts are invisible to duplicate control

Duplicate control compares normalised question text, span offsets and span text. Two provider libraries documenting the same operational behaviour share none of those, so the same fact can enter the benchmark twice from two SDKs. GOLD-B005-11 and GOLD-B006-06 both state that a base-URL environment variable overrides the region-derived Bedrock endpoint.

**Fix:** Compare candidates on their (subject, relation, object) triple, normalised, in addition to text and offsets. Batch 006 records that triple on every candidate, so the material for the check now exists. Flag rather than auto-drop: two libraries genuinely differing in behaviour is a real case, and only a reviewer can tell the two apart.

#### F. compound single-span facts are labelled by their first verb

The predicate lane picks a frame from the first matching verb and takes the reasoning type from that frame. Three of the nine exported candidates were relabelled by the owner: a requirement read as a configuration interaction, a compatibility statement read as an exact lookup, and a migration note read as a configuration interaction. The evidence was right in every case; the taxonomy was not.

**Fix:** Classify from the whole sentence rather than from the matched verb: a span naming a support status, a version or a migration is a lifecycle case whatever its verb, and `configuration_interaction` should require two settings that bear on each other rather than one requirement with two identifiers in it.

#### G. questions inherit the breadth of their frame, not of their evidence

'What does X reject?' and 'What does X default to?' ask for a complete list. The evidence gives one item, usually scoped to named models or surfaces. Three candidates needed rescoping, and in two of them the scope qualifier was not even in the critical strings, so nothing checked it.

**Fix:** When the source sentence carries a model, platform or surface qualifier, the question must carry it too and it must appear in the critical strings. Add a generation gate: a question whose evidence is scoped and whose wording is not, does not export.

## 9. Modules built

Each exists because a specific candidate got through a gate that should have caught it. Every one carries a regression test built from that candidate, not from an invented example — a test written from an imagined case tests the rule the author had in mind, and these rules exist because the author's rule missed the real thing.

| module                                      | what it checks                                            | added           | because of                            |
|---------------------------------------------|-----------------------------------------------------------|-----------------|---------------------------------------|
| <code>rag_v1.gold.scoping</code>            | bare definition bullets, asked of every span              | batch 006       | GOLD-B005-01                          |
| <code>rag_v1.gold.relations</code>          | source and question triples; direction check              | batch 006       | GOLD-B005-10                          |
| <code>rag_v1.gold.questionform</code>       | question form must match evidence form                    | batch 006       | GOLD-B005-08, -18                     |
| <code>rag_v1.gold.normalisation</code>      | markdown link stripping (3 link shapes)                   | batch 006       | GOLD-B005-15                          |
| <code>rag_v1.gold.defects</code>            | one defect shape across six renderers                     | batch 006       | three renderers crashed identically   |
| <code>rag_v1.gold.bridge_equivalence</code> | a bridge entity must mean the same thing in both spans    | batch 005       | max_tokens near-miss                  |
| <code>rag_v1.gold.mining_v5</code>          | interaction, constraint, lifecycle and ambiguity miners   | batch 005       | question subject ≠ fact subject       |
| <code>rag_v1.gold.multihop</code>           | dependency-first chain search                             | batch 005       | batch 004's 559-pair all-pairs search |
| <code>rag_v1.gold.authoring</code>          | question builders, including the batch-006 predicate lane | batches 005-006 | 2482 facts no builder could reach     |
| <code>rag_v1.gold.eligibility</code>        | the deterministic holdout gate (6 conditions)             | batch 005       | required_evidence_declared added      |

## 10. Heading parser audit

**44 of 5857 parsed headings (0.75%)** read as ordinary prose rather than as a label, across 15 of 202 documents. 82 were suspicious on at least one rule.

| why a heading was flagged                  | headings |
|--------------------------------------------|----------|
| starts lowercase                           | 30       |
| ends in a sentence-final period            | 21       |
| contains a comma-separated 'or'/'and' list | 16       |
| longer than 12 words                       | 14       |
| ends in a comma, semicolon or colon        | 13       |

44 of 5857 parsed headings (0.75%), in 15 of 202 documents. That is isolated. It does not justify a parser experiment on its own, and the batch-006 rule — never trust `section_path` for scope — is the cheaper fix.

**Nothing was rewritten.** No heading changed, no document was reparsed into storage, no evidence anchor moved. A closed case approved against a bad `section_path` was approved against its *evidence*; the path is metadata beside it. What changed is a rule: `section_path` is never trusted for claim scope, and a candidate's exact evidence must carry the scope its claim needs.

## 11. Batch 007 — preregistered, not generated

PREREGISTERED — no batch-007 candidate has been generated

### THE LINE

**The authoring model may change WORDING. It may not change MEANING. It authors the QUESTION; it never invents the FACT.**

Batch 007 introduces **controlled evidence-grounded question paraphrasing**: where a deterministic template cannot express an explicit evidence fact, a model may author the question. This is an **authoring** change. The evidence stays frozen and exact, the ground truth is still read out of the source, and every existing gate still runs.

### The order is the safeguard

1. frozen source evidence selected
2. literal source fact extracted
3. subject / relation / object recorded
4. atomic claims anchored
5. only then the natural question is paraphrased

**Never:** invent question → search for supporting evidence. Inventing a question and then hunting for evidence to support it is how a benchmark ends up testing what its author imagined rather than what the documentation says.

### The entailment self-check — any failure drops the candidate

| check                                                                | it fails when                                                                                    |
|----------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| <b>A</b> Does the exact evidence support the complete answer?        | any part of the answer is not derivable from the anchored spans                                  |
| <b>B</b> Does every atomic claim map to literal evidence?            | a claim has no span whose text supports it                                                       |
| <b>C</b> Did the question introduce any new condition?               | the question adds an 'if', 'when' or 'unless' the source lacks                                   |
| <b>D</b> Did it broaden model/provider/platform scope?               | the source names a model, provider, platform or surface and the question drops or generalises it |
| <b>E</b> Did it reverse the relation?                                | the question's subject is the source's object and vice versa, and the relation is not symmetric  |
| <b>F</b> Did it introduce a causal claim absent from the source?     | the answer explains why, and the source only says what                                           |
| <b>G</b> Could the answer be verified using only the exact evidence? | verifying it needs the surrounding document, a heading, or outside knowledge                     |

Every paraphrased candidate also stores its `source_fact_literal` and its subject/relation/object beside the authored question, so a reviewer can see the gap between fact and question and disagree with it.

### Answer conservatism

The question may be naturalised more aggressively than the answer. Prefer answers close to the source wording. Source says *A overrides B* → answer *A overrides B*, **not** *A overrides B because this improves reliability*. The reason is not in the source. An answer that explains is an answer that asserts something the evidence cannot check.

### A pilot gates the lane

**10 spans** that failed batch 006 *only* because no builder could express them — not spans that failed a semantic gate, which failed for reasons paraphrasing does not fix. Run through the paraphraser and every semantic check, then **independently reviewed** before

the lane may scale.

| criterion                            | threshold  |
|--------------------------------------|------------|
| independently judged factually sound | ≥ 8 of 10  |
| unsupported claims                   | 0          |
| relation-direction reversals         | 0          |
| scope broadening                     | 0          |
| wording cleanup needed               | acceptable |

**If it fails:** Do not scale the paraphrasing lane. Revise the authoring contract first and re-pilot.

Every existing gate still runs

| gate                                | implemented in              | behaviour                                                            |
|-------------------------------------|-----------------------------|----------------------------------------------------------------------|
| bare definition scope               | rag_v1.gold.scoping         | every span, independently                                            |
| critical anaphora                   | rag_v1.gold.anaphora        | blocks; noncritical flags                                            |
| subject / relation direction        | rag_v1.gold.relations       | REVERSED and SUBJECT_MISMATCH both drop                              |
| question form matches evidence form | rag_v1.gold.questionform    | negative-as-positive and truncated predicates drop                   |
| duplicate detection                 | scripts/export_batch_007.py | question text, span offsets, span text — and now the relation triple |
| critical strings                    | rag_v1.gold.normalisation   | every one must be literally inside its own span                      |
| evidence hashes                     | scripts/export_batch_007.py | each span hashes to its own text                                     |
| scope self-containment              | rag_v1.gold.scoping         | section_path is never claim scope                                    |
| example-code restriction            | scripts/export_batch_007.py | a sample configuration is not a rule                                 |
| evidence size                       | scripts/export_batch_007.py | <500 preferred, 1000 soft cap, 1500 hard cap                         |
| provider / model scope              | scripts/export_batch_007.py | a scoped source needs a scoped question                              |
| holdout eligibility                 | rag_v1.gold.eligibility     | deterministic, run after owner approval                              |

**THE ARITHMETIC, WHICH IS NOT A PLAN**

The project holds **90** and the target is **150**, leaving **60**. Batch 007 targets **35-40** candidates, which at the six observed acceptance rates lands between **117** and **130** eligible cases — **short of 150**. More than one batch will be needed, and that is not a reason to approve a candidate that should not be approved.

## 12. Tests and invariants

The suite is **720 passed**. Ten test files cover GOLD-001 specifically, including regression tests built from `GOLD-B005-01` , `-08` , `-10` , `-11` , `-15` and `-18` , and from batch 006's duplicate against `GOLD-B005-11` .

### What the tests hold

- **Closed-batch immutability.** Every closure hash is recomputed from the records it covers. An edit to a closed batch fails the suite.
- **No retrieval leakage.** `retrieval_was_not_run` is asserted on every batch and every record, and each record is checked for retrieval-derived fields.
- **Frozen system hashes.** SYSTEM-A `9afcb5b7...` and SYSTEM-B `304c3509...` are compared against the frozen constants.
- **Evidence integrity.** Every span hashes to its own text; every critical string is literally inside its own span.
- **Taxonomy changes move no evidence.** Offsets, text and hashes are compared before and after every relabelling.
- **Nothing claims verification it lacks.** Generation artifacts must carry no decision; only the owner appears as a reviewer.
- **The paraphrasing contract.** Batch 007 cannot broaden scope, add conditions, reverse relations or introduce causal claims without a test failing.

### Invariants, unchanged across all six batches

- No retrieval system has been run against any GOLD candidate at any point. `retrieval_was_not_run = true` ; `systems_executed = []` . No question was selected, ordered or worded because of retrieval difficulty.
- SYSTEM-A and SYSTEM-B remain frozen and unexecuted; the corpus snapshot, chunks, embeddings, fusion, DOC-C, top-k, routing, prompts and reranking are unchanged. All of this has been evaluation-authoring work only.
- No holdout is frozen and no validation split is frozen.
- Closed batches are never modified. Repairs live beside a generation artifact, never inside it; a new discovery about old tooling becomes an audit, an erratum or a future overlay, never a silent mutation.
- Claude's self-review is authoring, not verification. It has never been presented as independent verification, and no AI may set `human_verified` .

## 13. Commits and artifacts

| commit  | date       | subject                                                                          |
|---------|------------|----------------------------------------------------------------------------------|
| e65912a | 2026-08-25 | GOLD-001 batch 006 closed at 8/9; batch 007 preregisters controlled paraphrasing |
| 78f47ef | 2026-08-24 | GOLD-001 batch 006: four preregistered fixes, and 9 candidates not 28            |
| 6f0588d | 2026-08-24 | GOLD-001: close batch 005 at 15/19, record batch-006 inputs                      |
| 735244a | 2026-08-23 | GOLD-001 batch 005: source-integrity review and owner QC packet                  |
| 560b782 | 2026-08-23 | GOLD-001 batch 005: generation PDF, and make the PDF a standing deliverable      |
| 8bedfb1 | 2026-08-23 | GOLD-001 batch 005: two-lane coverage, and 19 candidates rather than 30          |
| f4c7d4e | 2026-08-23 | GOLD-001 batch 004: closure PDF                                                  |
| f7473d1 | 2026-08-23 | GOLD-001 batch 004: owner decisions imported, batch closed at 14/15              |
| 53b940c | 2026-08-23 | GOLD-001 batch 004: review-preparation PDF for independent verification          |
| cf67916 | 2026-08-21 | GOLD-001 batch 004: internal source-integrity review and owner QC packet         |
| 911a585 | 2026-08-21 | GOLD-001 batch 004: shareable PDF of the generation results                      |

| commit  | date       | subject                                                                                      |
|---------|------------|----------------------------------------------------------------------------------------------|
| 2daaa9e | 2026-08-21 | GOLD-001 batch 004: genuine multi-hop, mined against a check that fails it                   |
| 7067c40 | 2026-08-21 | GOLD-001: approve B003-04, close batch 003 at 20/0, project total 53 holdout-eligible        |
| 4bc36ad | 2026-08-20 | GOLD-001: import the batch-003 owner decisions and rewrite B003-04 without touching evidence |

Where each figure in this document comes from

- experiments/GOLD-001/GOLD-001-eligibility-status.json — project totals and the per-batch eligibility figures.
- experiments/GOLD-001/GOLD-001-batch-00N-closure.json (N = 1...6) — acceptance, closure hashes, repairs, overrides, shortfalls, multi-hop results.
- evals/review/gold\_review\_batch\_00N\*.json — the candidate records themselves, and batch 001's v2 overlay.
- experiments/GOLD-001/b00N-review-decisions.json — the review findings and the generator defects each batch recorded.
- experiments/GOLD-001/GOLD-001-heading-parser-audit.json — the heading figures.
- experiments/GOLD-001/GOLD-001-batch-007-preregistration.json — the batch-007 contract.

Generated by scripts/build\_gold001\_complete\_record.py. Every figure is read from the artifacts listed above at build time; none is typed into this document. The build refuses to run if any batch is missing a closure, if a closure's totals disagree with its records, if the eligibility gate re-run disagrees with any closure, if the project totals disagree with the sum of the batches, if any batch records a retrieval run, if a frozen system hash has moved, if a batch-007 artifact exists, or if the page would claim batch 007 reaches the project target. Raw provider documentation is not redistributed; the corpus is referenced by snapshot id and cases by character offset.